LoRa Beacon Detection with RTL-SDR

Reference guide to recreate the experiment from scratch · SIH26050 anti-drone node PoC · Parts 1 and 2 (RTL-SDR + LoRa beacon)

What this experiment proves

A LoRa chip pretends to be a drone's radio link. An RTL-SDR radio stick records the air around it. The **rtl-drone-rf** code reads the recording and flags the signal as drone-like. Chain tested end to end on 25 Oct 2026:

```
LoRa chip -> air -> antenna -> RTL-SDR stick -> SDR++ recording (.wav) -> rf_scanner.py -> threat report
```

Which laptop does what

Role	Laptop	Why
Receiver (radio stick + SDR++ + scanner code)	**Windows**	The rtl-drone-rf repo was built and tested on Windows. Driver (Zadig) and SDR++ setup on Windows is proven.
Transmitter (LoRa chip + beacon script)	**Mac**	Only needs Python and a USB serial port. Recent macOS has the CH340 driver built in, so nothing extra to install. (Ubuntu also works — see Appendix A.)

Contents

1. Parts list
2. Part A — Receiver setup (Windows)
3. Part B — Transmitter setup (Mac)
4. Part C — Running the experiment
5. Expected results
6. Troubleshooting (every problem we hit)
7. Known limitations
8. Quick reference card

Appendix A — Ubuntu as transmitter

1. Parts list

Part	Used for	Notes
Nooelec NESDR SMArt v5 bundle (Robu)	Receiver	USB radio stick + 3 antennas. Covers about 100 kHz to 1.75 GHz.
Reyax RYLR998 LoRa module (Robu)	Transmitter	Works at 3.3 V only. Has a spring antenna on it — keep it on.
WEMOS CH340G USB-to-serial board (Robu)	Transmitter	Lets the laptop talk to the LoRa chip over USB. Has a yellow 5V/3.3V jumper.
Female-to-female jumper wires (4)	Transmitter	Both boards have pins, so wires need holes on both ends.
Capacitor, 10–100 µF (optional)	Transmitter	Steadies the chip's power when it transmits. Striped side to GND.
ESP32 dev board (optional)	Transmitter	Backup power source for the chip if the CH340's 3.3 V is too weak.
Windows laptop + Mac	Both	One receives, one transmits.

2. Part A — Receiver setup (Windows)

Do this once. Every command below goes in **PowerShell** (Start menu → search "PowerShell").

A1. Check Python and Git are installed

```
python --version
git --version
```

Both must print a version. Python must be 3.10 or newer. If missing, install from python.org (tick "Add to PATH") and git-scm.com.

A2. Download the code and install its libraries

```
cd ~\Documents
git clone https://github.com/Megh089/rtl-drone-rf.git
cd rtl-drone-rf
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

- **.venv** is a private folder that keeps this project's libraries separate from the rest of the laptop. When active, the prompt starts with **(.venv)**. You must activate it every time you open a new PowerShell window.
- If activate gives a red "running scripts is disabled" error, run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
- If install fails on **pyrtlsdr**, delete that line from requirements.txt and re-run. The code does not use it.

Folder location: the code lives at `C:\Users\<you>\Documents\rtl-drone-rf`. If File Explorer's "Documents" doesn't show it, OneDrive has hijacked the shortcut. Paste the full path into the address bar, or run `explorer .` from inside the folder in PowerShell.

A3. Test the code with fake signals (no hardware)

```
python generate_test_signals.py
python rf_scanner.py synthetic_drone_telemetry.wav 433900000
```

Pass: ends with **SCAN COMPLETE**, threat scores around 50–60, two report files saved. Yellow "InconsistentVersionWarning" lines are harmless (model saved with sklearn 1.9.0, you have 1.9.1). It reports 4 signals from 1 fake transmitter — the loudest (~-45 dB) is real, the rest are side bumps from the sharp on/off switching.

A4. Install the radio stick driver (Zadig)

1. Plug in the RTL-SDR stick and wait for Windows to finish "setting up".
2. Download Zadig from **zadig.akeo.ie** and run it (no install needed).
3. Click **Options → List All Devices**.
4. In the dropdown pick **NESDR SMArt v5**. (On other sticks it may be called "Bulk-In, Interface (Interface 0)".)
5. Check the USB ID boxes read **0BDA** and **2838**. If not, you picked the wrong device — stop.
6. Make sure the box right of the green arrow says **WinUSB**.
7. Click **Install Driver** (or **Replace Driver** — same result). Wait for success.

Do NOT pick "Wireless Receiver" (your mouse/keyboard dongle, USB ID 046D), the webcam, or Bluetooth. Replacing their driver stops them working.

A5. Install and set up SDR++

1. Search "SDR++ github releases", download the **windows x64 zip**, unzip it, run **sdrpp.exe**.
2. Apply these settings once:

Setting (left panel)	Value	Why
Source	RTL-SDR, device: Nooelec NESDR SMArt v5	Your stick
Sample rate	2.4 MHz	Records a 2.4 MHz-wide slice. The code expects this.
Gain	~30 dB for FM, ~15 dB for the LoRa test	0 dB = deaf. Too high with the chip nearby = overloaded.
Recorder mode	**Baseband** (not Audio)	Baseband = raw radio data the code needs. Audio = just sound.
Recorder folder ("..." button)	C:\Users\<you>\Documents\rtl-drone-rf	Saves straight into the code folder — no copying.
Container / Sample type	WAV / **Int16**	The code's file reader assumes Int16.

Press ▶ to start (it turns into ■ when running). Working = a wiggly line around -90 dB and scrolling blue waterfall.

A6. Sanity check on FM radio

1. Tune SDR++ to a local FM station (e.g. **91.100.000**). Gain ~30 dB, telescopic antenna fully out. You should hear it.
2. Click **Record**. The line under the button should change from "Idle" to a counting timer. Wait 5 s, click **Stop**. (5 s ≈ 50 MB.)
3. Run the code on it (use Tab to auto-complete the file name):

```
python capture_spectrum.py baseband_91100000Hz_<...>.wav 91100000
python rf_scanner.py      baseband_91100000Hz_<...>.wav 91100000
```

Pass: graph shows a big hump at 91.1 (about 150 kHz wide) plus smaller humps for nearby stations. Scanner says **NONE (GREEN)**, score ~4, "not in a known drone band". The edges of the graph sagging is the stick's own filter — ignore it.

3. Part B — Transmitter setup (Mac)

B1. Set the jumper and wire it up

> **FIRST:** move the yellow jumper on the CH340G board so it joins **VCC** and **3V3**. It ships on 5V. 5 V will damage the LoRa chip. Do this before plugging anything into USB.

CH340G board		RYLR998 chip	Note
3V3 (or VCC)	→	VDD	Power
GND	→	GND	Ground
TXD	→	**RXD**	Crossed: one board's "talk" goes to the other's "listen"
RXD	→	**TXD**	Crossed
—		NRST	Leave unconnected

- Optional capacitor across the chip's VDD and GND: **striped side to GND**, long leg to VDD.
- If the chip answers commands but never shows up on the receiver, power is the suspect — see Troubleshooting.

B2. Install Python and the serial library

Open **Terminal** (Cmd+Space → "Terminal").

```
python3 --version
```

If macOS offers to install "command line developer tools", accept and wait. Then make a private library folder and install pyserial:

```
mkdir ~/lora && cd ~/lora
python3 -m venv .venv
source .venv/bin/activate
pip install pyserial
```

Every new Terminal window: `cd ~/lora && source .venv/bin/activate`

B3. Find the chip's port

Plug the CH340G into the Mac, then:

```
ls /dev/cu.*
```

Look for a name like **/dev/cu.usbserial-XXXX** or **/dev/cu.wchusbserialXXXX**. That's your port. Tip: run the command before and after plugging in — the new entry is the board. Use the **cu.** name, not the **tty.** one. If nothing new appears on an older macOS, install the CH34x driver from wch-ic.com.

B4. Talk to the chip (lora.py)

Why a script and not a normal serial terminal: terminals like picocom or screen send each key the moment you press it. The chip wants a whole command in one go, so it replies **+ERR=1** / **+ERR=4**. This script sends whole lines.

Create **lora.py** in ~/lora (e.g. `nano lora.py`, paste, Ctrl+O, Enter, Ctrl+X). **Change PORT to yours.**

```
import serial, time

PORT = '/dev/cu.usbserial-XXXX'   # <- change to your port
s = serial.Serial(PORT, 115200, timeout=0.5)

while True:
    cmd = input('> ')
    s.write((cmd + '\r\n').encode())     # send whole line + the ending the chip needs
    time.sleep(1.0)                       # wait for the reply
    print(s.read(200).decode(errors='ignore'))
```

```
python3 lora.py
```

Type **AT** and Enter. **Pass: +OK**. Ctrl+C quits.

B5. The beacon script (beacon.py)

Sets the chip up every time it starts (so you never depend on old settings), prints the chip's replies, then sends HELLO about twice a second until Ctrl+C.

```
import serial, time

PORT = '/dev/cu.usbserial-XXXX'   # <- change to your port
s = serial.Serial(PORT, 115200, timeout=1)

def cmd(c):
    s.write((c + '\r\n').encode())
    time.sleep(0.5)
    return s.read(200).decode(errors='ignore').strip()

print('band     :', cmd('AT+BAND=866000000'))      # 866 MHz (legal in India)
print('params   :', cmd('AT+PARAMETER=9,7,1,12'))  # SF9, 125 kHz, CR 4/5, preamble 12
print('power    :', cmd('AT+CRFOP=0'))             # 0 dBm = lowest, enough for one room
print('band now :', cmd('AT+BAND?'))

n = 0
while True:
    print(n, cmd('AT+SEND=0,5,HELLO'))            # 5 = number of letters in HELLO
    n += 1
```

Pass: first three lines say **+OK**, "band now" says **866000000**, then every line says **+OK**.

> **Why 866 MHz and not 868:** in India the licence-free band is 865–867 MHz. 868 and 915 are not free to use here. On our first try the band setting silently didn't stick and the chip sat on its default **915 MHz** — always check the "band now" line.

4. Part C — Running the experiment

1. **Windows:** open SDR++, press ▶. Tune to **865.800.000**. Gain about **15 dB**. Recorder on Baseband / Int16 / code folder.
2. **Mac:** `cd ~/lora && source .venv/bin/activate && python3 beacon.py`
3. **Windows:** you should see short stripes appear in the waterfall at **866.0** (just right of the red centre line), about two per second.
4. Click **Record**. Confirm the timer is counting. Wait **10 seconds**. Click **Stop**.
5. **Mac:** Ctrl+C to stop the beacon.
6. **Windows PowerShell** (in the code folder, with (.venv) active):

```
dir *.wav
python rf_scanner.py baseband_865800000Hz_<...>.wav 865800000
```

The number at the end must be the frequency SDR++ was actually tuned to, in Hz. It's in the file name (baseband_**865800000**Hz_...). Don't rename recordings — the name is your record of where you were tuned. Giving the wrong number shifts every reported frequency by the difference.

Why tune to 865.8 and not 866.0: RTL-SDR sticks always show a fake spike at the exact frequency they're tuned to. Tuning 0.2 MHz below puts your real signal off to the side, away from the spike.

5. Expected results

Test	What you should get	What we got (25 Oct 2026)
Fake signals (A3)	SCAN COMPLETE, scores ~50–60	4 detections, top 61/100 HIGH at 434.0 MHz
FM radio (A6)	One big hump at the station, GREEN	91.1 MHz, 143 kHz wide, score 4/100 NONE (GREEN)
LoRa beacon (C)	Signal at ~866.0 MHz, ~125–150 kHz wide, LoRa-family fingerprint, score ~50–60, ML label probably wrong	866.0 MHz, 136 kHz, TBS Crossfire match 80/100, scores 53 and 62. ML said FM_BROADCAST / DRONE_VIDEO (wrong, as expected).

Note: our recorded run was tuned to 866.0 (not 865.8), so the signal sat on the centre spike and was reported as **two** signals (865.994 and 866.052, both ~137 kHz wide) — one LoRa signal counted twice. Tuning to 865.8 as in Part C should give a single detection; that run has not been recorded yet.

6. Troubleshooting

Symptom	Cause	Fix
SDR++ graph and waterfall completely black, no line at all	Not running, or stick froze	Press ▶. If running but still blank: ■ stop, unplug/replug stick, ▶. Still blank: restart SDR++.
Flat line, can't hear or see FM	Gain at 0 or antenna loose	Gain to ~30 dB, screw antenna on, pull it out fully.
Record clicked but no file appears	Not actually recording / unknown save folder	Status under Record must count up. Set the folder with "..." to the code folder.
Can't find rtl-drone-rf in File Explorer	OneDrive "Documents" is a different folder	Paste the full path into the address bar, or run explorer . in PowerShell.
Chip replies +ERR=1 / +ERR=4 to every key	Terminal sends one key at a time	Use lora.py, which sends whole lines.
Chip answers AT but nothing shows on receiver	(a) wrong frequency, or (b) weak power: chip restarts when it transmits	(a) check AT+BAND?. (b) if AT+SEND gives +OK then +READY, feed VDD from an ESP32's 3V3 pin instead, all GNDs joined.
Stripes appear at 915 MHz, not 866	Band setting didn't stick (915 is the default)	Run beacon.py — it sets 866 every time and prints the check.
"No such file or directory" for the port	Typo, or the board got a new port name after replug	Re-run the ls command, update PORT. Linux: /dev/ttyUSB0 (no slash after tty).
Scanner reports signals ~0.2 MHz off	Wrong centre frequency given to the scanner	Use the frequency in the recording's file name.
Scanner finds 2 signals for one transmitter	Signal sat on the centre spike, or side bumps	Tune 865.8 when recording 866.0.

7. Known limitations

- **The ML guesser has never seen LoRa.** It's trained only on fake signals and has no LoRa class, so its label is unreliable. The frequency and fingerprint checks still catch the signal. Fix later: add a LoRa (chirp) class to train_rf_classifier.py and use real recordings as training data.
- **"Proximity" is a rough guess.** It's based on loudness, which also depends on gain and antenna. It said the FM tower was "within a few hundred metres" — it's kilometres away.
- **Side bumps** next to strong signals can be counted as separate signals.
- **Can't tell LoRa drone radios apart.** Crossfire, ExpressLRS and FrSky R9 use the same kind of signal; the matcher lists all plausible ones.
- **Receive only.** The stick can't transmit; the LoRa chip can only send LoRa packets. No jamming/spoofing.
- **Offline, not real-time.** Record first, analyse after.

8. Quick reference card

Settings

Item	Value
LoRa frequency	866.000 MHz (AT+BAND=866000000)
LoRa settings	SF9, 125 kHz bandwidth, CR 4/5, preamble 12 (AT+PARAMETER=9,7,1,12)
LoRa power	0 dBm (AT+CRFOP=0)
Serial	115200 baud, every command ends with \r\n
SDR++ tuning for LoRa test	865.800 MHz, 2.4 MS/s, gain ~15 dB, Baseband, WAV, Int16
Scanner command	python rf_scanner.py <file>.wav <tuned frequency in Hz>

RYLR998 commands used

Command	Does
AT	Am I connected? Replies +OK
AT+BAND=866000000 / AT+BAND?	Set / read frequency (Hz)
AT+PARAMETER=9,7,1,12 / AT+PARAMETER?	Set / read LoRa settings
AT+CRFOP=0 / AT+CRFOP?	Set / read power (0–22 dBm)
AT+SEND=0,5,HELLO	Send "HELLO" (5 letters) to address 0

Chip replies

Reply	Meaning
+OK	Command accepted
+READY	Chip just started up. Seen right after AT+SEND = it restarted mid-transmit (power problem).
+ERR=1	Command arrived without the line ending
+ERR=4	Unknown command (e.g. an empty line)

Every-session start-up

```
WINDOWS (receiver)                          MAC (transmitter)
cd ~\Documents\rtl-drone-rf                 cd ~/lora
.venv\Scripts\activate                       source .venv/bin/activate
open SDR++, press ▶, tune 865.8             python3 beacon.py
```

Appendix A — Ubuntu as transmitter

This is what we actually used on 25 Oct. Same scripts; only the port and setup differ.

```
ls /dev/ttyUSB*                     # expect /dev/ttyUSB0
sudo apt remove brltty              # only if the port doesn't appear (it steals CH340 boards)
sudo usermod -aG dialout $USER      # permission to use the port; log out and back in after
sudo apt install python3-serial
python3 beacon.py                   # with PORT = '/dev/ttyUSB0'
```

If you skip the log-out, prefix the run with sudo. If the port vanishes after a replug, re-run ls — it may now be ttyUSB1.